

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 — ARCHITECTURE DESIGN REPORT

Intelligent Supply Chain Risk Prediction and Vendor Management Platform

Field	Details
Course Code / Title	CSA1013 — Software Engineering
CO Assessed	CO2 — Requirements analysis, scalable architecture design, and feature prioritization
Assigned Problem	Intelligent Supply Chain Risk Prediction and Vendor Management Platform
Student Name	Ch B Vinay
Registration Number	192425148
Date	05/08/2026

Code of Conduct

I Ch.B.Vinay (192425148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Ch B Vinay*

1. Analysis of System Requirements

1.1 System Objectives

The problem statement identifies that global vendor networks expose organizations to risk that is broader than late delivery alone: shifting tariffs, ESG/labor-compliance gaps, geopolitical instability, and vendor financial distress, each owned by a different data source and often a different internal team. The proposed platform must therefore treat vendor, logistics, trade/tariff, ESG, and geopolitical data as distinct, independently owned data domains; combine them into a composite risk score; simulate trade-policy cost impact; verify compliance in a tamper-evident way; and recommend diversification - ultimately improving procurement resilience and auditability.

1.2 Functional Requirements

The functional requirements below were derived directly from the stated objectives (domain data ingestion, composite scoring, simulation, diversification, and compliance auditing):

ID	Functional Requirement
FR-1	Ingest vendor, contract, and transaction data from the ERP/procurement domain.
FR-2	Ingest carrier/logistics performance and routing data from the logistics domain.
FR-3	Ingest tariff and trade-policy data from the trade/tariff domain.
FR-4	Ingest ESG/sustainability ratings from the ESG domain.
FR-5	Ingest geopolitical event data from the geopolitical risk domain.
FR-6	Compute a composite vendor risk score on demand using an elastic, serverless scoring function.
FR-7	Simulate tariff/trade-policy impact on landed cost per vendor/route via an on-demand simulation function.
FR-8	Score supplier concentration/diversification risk and recommend alternate sourcing.

1.3 Non-Functional Requirements and Key Quality Attributes

Because the platform combines independently governed data domains and must audit every compliance decision, its non-functional requirements are architecture-defining rather than incidental:

Quality Attribute	Description
Scalability	Must elastically absorb bursty, on-demand workloads (e.g., many simultaneous tariff simulations) without over-provisioning idle capacity.
Performance	Composite risk scores and simulation results should return within a few seconds of a request under normal load.
Reliability	Risk scores and compliance records must remain accurate and available even if a subset of domain data products or functions fail.
Availability	Core dashboard and compliance-recording services should be available 24/7 (target 99.9% uptime) since procurement decisions depend on them.
Security	Vendor financial, ESG, and compliance data must be authenticated, encrypted, and access-controlled per domain.
Maintainability	Each data domain and compute function must be independently updatable without full system downtime.
Auditability	Every compliance verification event must be recorded immutably with a verifiable timestamp and actor identity.
Interoperability	Must integrate heterogeneous domain APIs (ERP, carrier, trade/tariff, ESG, geopolitical) through a single governed access layer.

2. Selection of Suitable Software Architecture

2.1 Selected Architectural Style

Hybrid Architecture: Data Mesh with a Serverless Compute Layer and a Blockchain-Backed Audit Ledger.

The platform combines three complementary styles: (1) a Data Mesh, in which each data source - vendor/ERP, logistics, trade/tariff, ESG, and geopolitical - is treated as a domain-owned data product published through a governed API/data contract rather than centralized into one team's pipeline; (2) a Serverless Compute Layer for elastic, on-demand workloads such as composite risk scoring, tariff simulation, and diversification analytics, which are bursty and do not need to run continuously; and (3) a small set of Core Persistent Microservices (risk engine/model store, vendor scoring, compliance/onboarding, analytics) for stateful business logic that benefits from always-on availability, backed by an immutable, blockchain-based ledger for compliance audit events.

2.2 Justification

- The five data sources (ERP, carrier API, trade/tariff database, ESG service, geopolitical feed) are owned by different external providers and internal teams with different update cadences and governance needs - a Data Mesh lets each domain publish and version its own data product instead of forcing everything through one brittle, centrally owned pipeline, which a monolithic data-warehouse-first design would not handle well.
- Composite scoring and tariff simulation are triggered on demand (e.g., when a procurement officer requests a what-if analysis) rather than continuously - a Serverless Compute Layer scales to zero between requests and elastically absorbs bursts (e.g., many simultaneous simulations during a trade-policy announcement), which is more cost-efficient than dedicating always-on compute to intermittent workloads.
- Compliance verification requires an immutable, independently verifiable record - a blockchain-backed ledger was chosen specifically for this sub-problem because it provides tamper-evidence and multi-party auditability that a conventional mutable database cannot offer on its own, while the rest of the platform continues to use conventional data stores for cost and performance reasons.
- A pure Event-Driven Microservices style (as might suit a continuous IoT-sensor workload) was considered but judged less appropriate here: most inputs in this scenario are periodic API pulls or

on-demand queries rather than a continuous physical sensor stream, so committing to an always-on message-broker backbone for every component would add operational overhead without a matching benefit. Event-style messaging is still used selectively between domains and core services where genuinely asynchronous.

This hybrid approach directly supports the required quality attributes - scalability, elastic cost efficiency, auditability, security, and maintainability - while remaining flexible enough to add new data domains or analytics functions over time.

3. Software Architecture Diagram

The diagram below illustrates the end-to-end structure of the platform: domain-owned data products, the unified API gateway, the elastic serverless compute layer, core persistent microservices, the blockchain-backed compliance ledger, and the application/presentation layer, along with the closed-loop vendor-status update written back to the ERP domain.

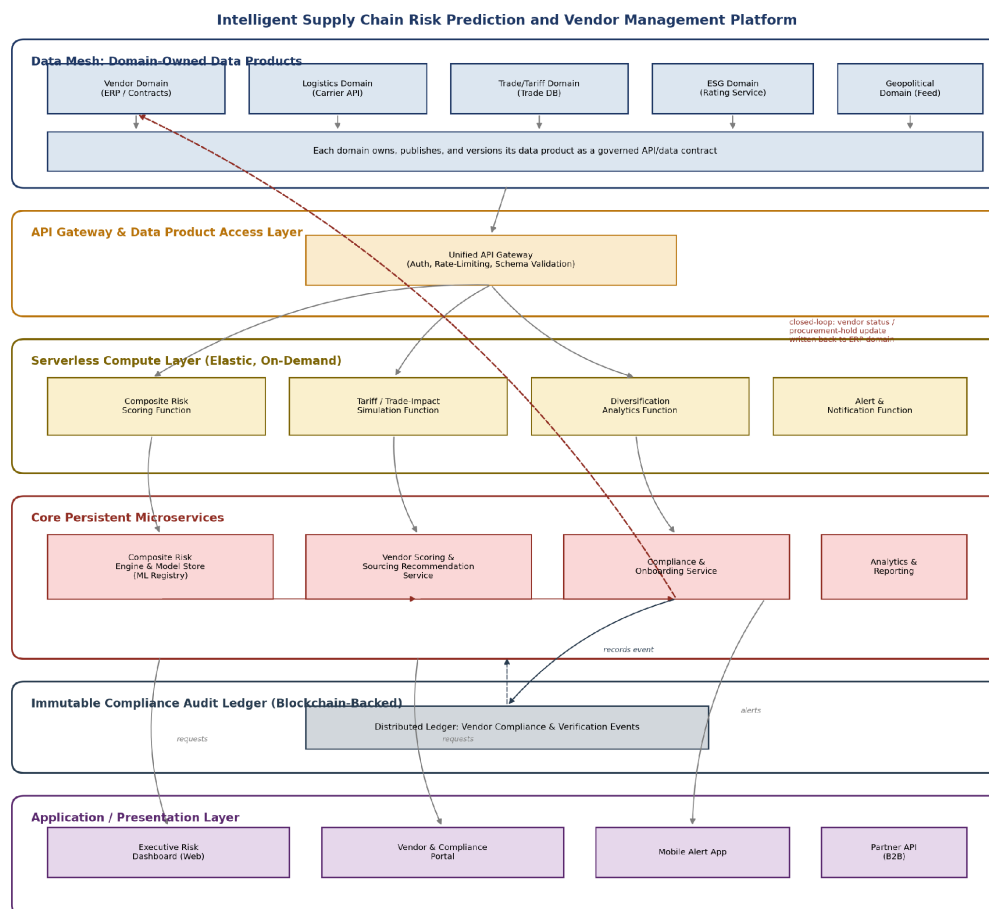


Figure 1: Data Mesh + Serverless architecture with blockchain-backed audit ledger for the Intelligent Supply Chain Risk Prediction and Vendor Management Platform.

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Purpose / Responsibility
Vendor Domain (ERP/Contracts)	Domain-owned data product publishing vendor master, contract, and transaction data.
Logistics Domain (Carrier API)	Domain-owned data product publishing shipment status and carrier performance data.
Trade/Tariff Domain (Trade DB)	Domain-owned data product publishing tariff rates and trade-policy changes.
ESG Domain (Rating Service)	Domain-owned data product publishing vendor ESG/sustainability ratings.
Geopolitical Domain (Feed)	Domain-owned data product publishing region-level geopolitical event data.
Unified API Gateway	Single governed access point for all domain data products; handles authentication, rate limiting, and schema validation.
Composite Risk Scoring Function	Serverless function that computes a vendor's composite risk score on demand from the latest domain data.
Tariff/Trade-Impact Simulation Function	Serverless function that simulates landed-cost impact of tariff/trade-policy changes per vendor/route.
Diversification Analytics Function	Serverless function that scores supplier concentration and diversification risk on demand.
Alert & Notification Function	Serverless function that evaluates thresholds and dispatches alerts/notifications.
Composite Risk Engine & Model Store	Core persistent microservice hosting the ML model registry and aggregated risk history.
Vendor Scoring & Sourcing Recommendation Service	Core persistent microservice that ranks vendors and recommends sourcing/diversification options.
Compliance & Onboarding Service	Core persistent microservice that runs compliance checks and writes verification events to the ledger.
Analytics & Reporting	Core persistent microservice that aggregates data for dashboards and reports.

Component	Purpose / Responsibility
Distributed Ledger (Blockchain-Backed)	Immutable store of vendor compliance and verification events, independently auditable.
Executive Risk Dashboard / Vendor & Compliance Portal / Mobile Alert App / Partner API	Presentation layer for internal stakeholders and, via a restricted B2B API, for vendor self-service.

4.2 Communication and Data Flow

- Each domain (vendor/ERP, logistics, trade/tariff, ESG, geopolitical) publishes its data as a versioned data product accessible only through the Unified API Gateway, which authenticates requests and validates schemas.
- When a user or scheduled job requests a risk view, the API Gateway invokes the relevant serverless function (Composite Risk Scoring, Tariff/Trade-Impact Simulation, or Diversification Analytics), which pulls the latest data directly from the domain products it needs.
- Serverless functions call into the Core Persistent Microservices layer to store computed scores, retrieve model parameters from the Model Store, and update vendor scoring/recommendations.
- The Compliance & Onboarding Service writes every verification event to the Distributed Ledger, giving compliance officers and auditors a tamper-evident history independent of the operational database.
- The Vendor Scoring & Sourcing Recommendation Service and Compliance & Onboarding Service feed the Analytics & Reporting service and the Alert & Notification function, and also write vendor-status/procurement-hold updates back to the Vendor Domain (ERP), closing the loop between prediction, verification, and the system of record.
- Stakeholders interact through the Executive Risk Dashboard, Vendor & Compliance Portal, and Mobile Alert App, which call back through the API Gateway for on-demand queries and simulations; a restricted Partner API extends limited self-service access to vendors.

5. Discussion of Quality Attributes

Attribute	How the Architecture Addresses It
Scalability	Serverless functions scale to zero between requests and burst elastically during demand spikes (e.g., simultaneous tariff simulations), while core microservices scale horizontally behind the API Gateway.
Security	The API Gateway centralizes authentication (OAuth2/JWT) and schema validation per domain; TLS encrypts all domain-to-gateway and gateway-to-service traffic; role-based access restricts who can query which domain or write to the ledger.
Performance	On-demand serverless functions avoid cold, batch-oriented processing for simulations, returning results within seconds; the composite risk engine caches recent scores to avoid recomputation for repeated dashboard views.
Reliability	Domain data products are independently owned and versioned, so a failure or delay in one domain (e.g., the ESG feed) degrades only that input rather than the whole composite score; core services are stateless where possible and can be restarted without data loss.
Maintainability	Each data domain, serverless function, and core microservice can be updated, redeployed, or replaced independently through its published contract, without touching the others.
Auditability	The blockchain-backed ledger provides an independently verifiable, tamper-evident record of every compliance verification event, satisfying regulatory and multi-party trust requirements that a conventional mutable log cannot fully guarantee.

6. Justification of Architectural Decisions

6.1 Rationale

The core rationale is that this platform integrates several independently governed, differently paced data sources (a system of record, a third-party carrier API, a trade database, an ESG rating service, and a geopolitical feed) and must support both on-demand analytical workloads (simulation, scoring) and an always-verifiable compliance record. Treating each data source as an owned data product (Data Mesh) avoids a single team becoming a bottleneck for every new integration, while serverless compute matches

the bursty, on-demand nature of scoring and simulation more efficiently than always-on services, and a dedicated ledger gives compliance events a guarantee that ordinary databases do not provide.

6.2 Trade-offs Considered

- Governance overhead vs. domain autonomy: a Data Mesh requires each domain team to maintain a stable, versioned data contract, which is more governance overhead than a single centralized data pipeline, but it was accepted because it avoids a central team becoming a bottleneck as new data sources (e.g., additional ESG providers) are added.
- Cold-start latency vs. cost efficiency: serverless functions can incur brief cold-start latency compared to always-on services, but this was accepted in exchange for not paying for idle compute between infrequent simulation requests; latency-sensitive dashboard queries are instead served from the always-on Core Persistent Microservices layer.
- Ledger overhead vs. auditability: writing every compliance event to a distributed ledger is slower and more resource-intensive than a plain database insert, but this was accepted specifically for compliance-critical events because the resulting tamper-evidence and independent verifiability materially reduce audit and regulatory risk.

6.3 Support for Future Enhancements and Long-Term Maintainability

- New data domains (e.g., a customs/trade-compliance feed or a sub-tier supplier registry) can be onboarded by publishing a new domain data product behind the API Gateway, without changing existing serverless functions or core services.
- New on-demand analytics (e.g., sustainability-linked financing risk, currency-fluctuation impact) can be added as additional serverless functions that read from existing domain products.
- Additional regions or vendor categories scale by adding domain data-product instances and letting the serverless layer absorb the added on-demand load, since core services remain stateless where possible.
- Independent versioning of each domain contract and each function/service supports continuous improvement of the composite risk model as more historical data accumulates.